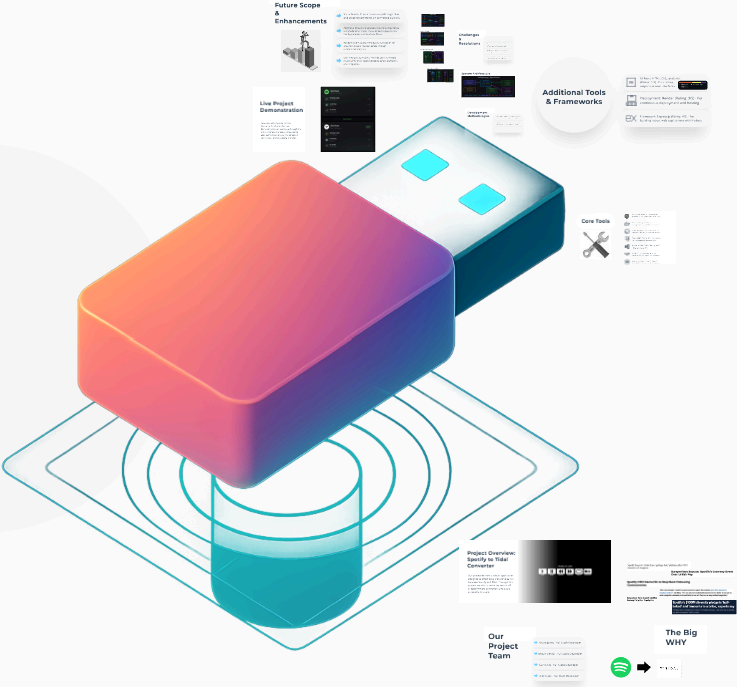
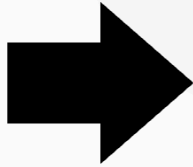


Spotify to Tidal Converter Project

Seamless Music Migration Solution



The Big WHY



Our Project Team

- ➡ Drake Jones - Full Stack Developer
- ➡ Ishaan Venkat - Full Stack Developer
- ➡ Curtis Liu - Full Stack Developer
- ➡ Alex Class - Full Stack Developer

MUSIC

Spotify boycott: Artists leave 'garbage hole' platform after CEO invests in AI weapons

Songwriters Boycott Spotify's Grammy Event Over Unfair Pay

Spotify CEO Daniel Ek to Step Back Following Controversies

"After some thought, I couldn't in good conscience support this initiative **given their approach to bundling royalties**," said Dillon. "It is very nice to be individually honored, but it is better for me and my entire songwriter community to be paid fairly for our art. There are no songs without songwriters."

Songwriters Opt to Boycott Spotify's Grammy Party Over Royalty Cuts

Spotify's \$100M diversity pledge is 'half-baked' and 'amounts to a bribe,' experts say

"The thing that's surprising here is that the CEO didn't anticipate – and clearly had no plan for – reacting to controversy," said one expert.

Project Overview: Spotify to Tidal Converter

Our project delivers a robust application designed to effortlessly transfer playlists between Spotify and Tidal. Through this project, we aim to make the switch off of Spotify more convenient and easily accessible to users.



Core Tools



Mocha + Chai: (Rating: 5/5) - Automatic Test Suite covering auth endpoints and server health checks



Docker: (Rating: 4/5) - For task management and workflow visualization.



GitHub (Rating: 5/5) - For version control, task tracking/management and collaborative coding.



PostgreSQL (Rating: 5/5) - For robust data storage and management.



Visual Studio Code (Rating: 4/5)
- Our primary IDE.



NodeJS + Express (Rating: 4/5) - For backend logic and server-side operations.



External APIs: Spotify Web API, Tidal API
(Rating: 5/5) - Necessary for data interaction.

Development Methodologies



Iterative Development: (Rating: 4/5) Continuous cycles of planning, building, and refining.



Agile Principles: (Rating: 4/5) Fostering flexibility, collaboration, and rapid response to change.

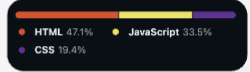


Peer Code Reviews: (Rating: 5/5) Ensuring code integrity, identifying bugs, and promoting best practices.

Additional Tools & Frameworks



UI Tools: HTML, CSS, JavaScript
(Rating: 5/5) - For crafting
responsive user interfaces.

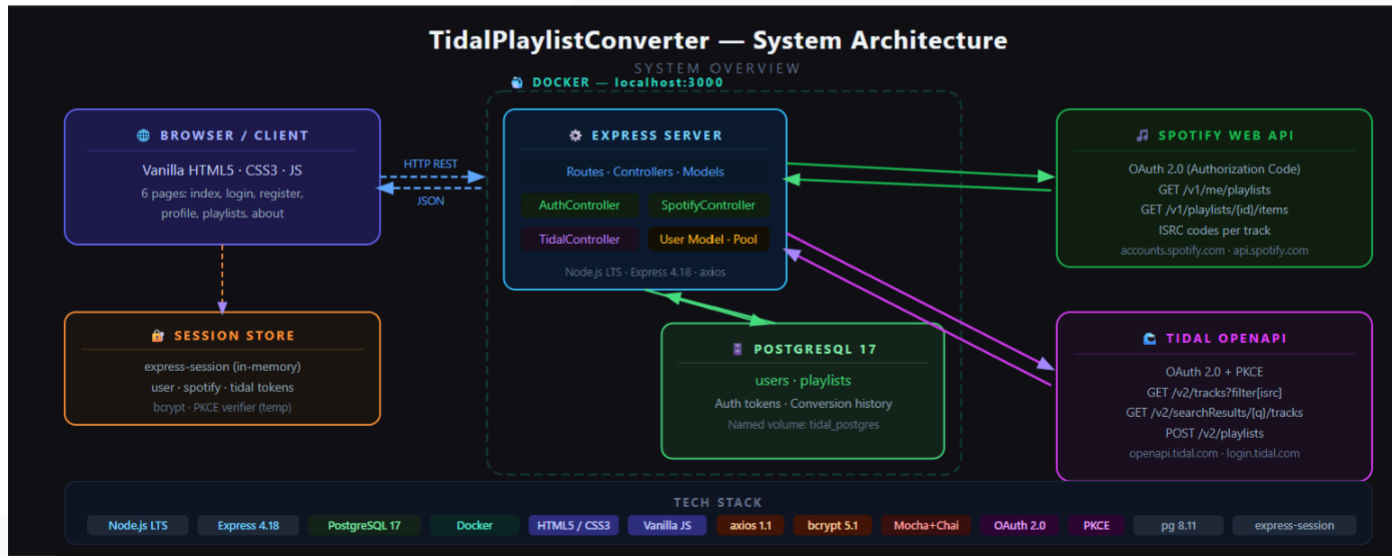


Deployment: Render (Rating: 3/5) - For
continuous deployment and hosting.

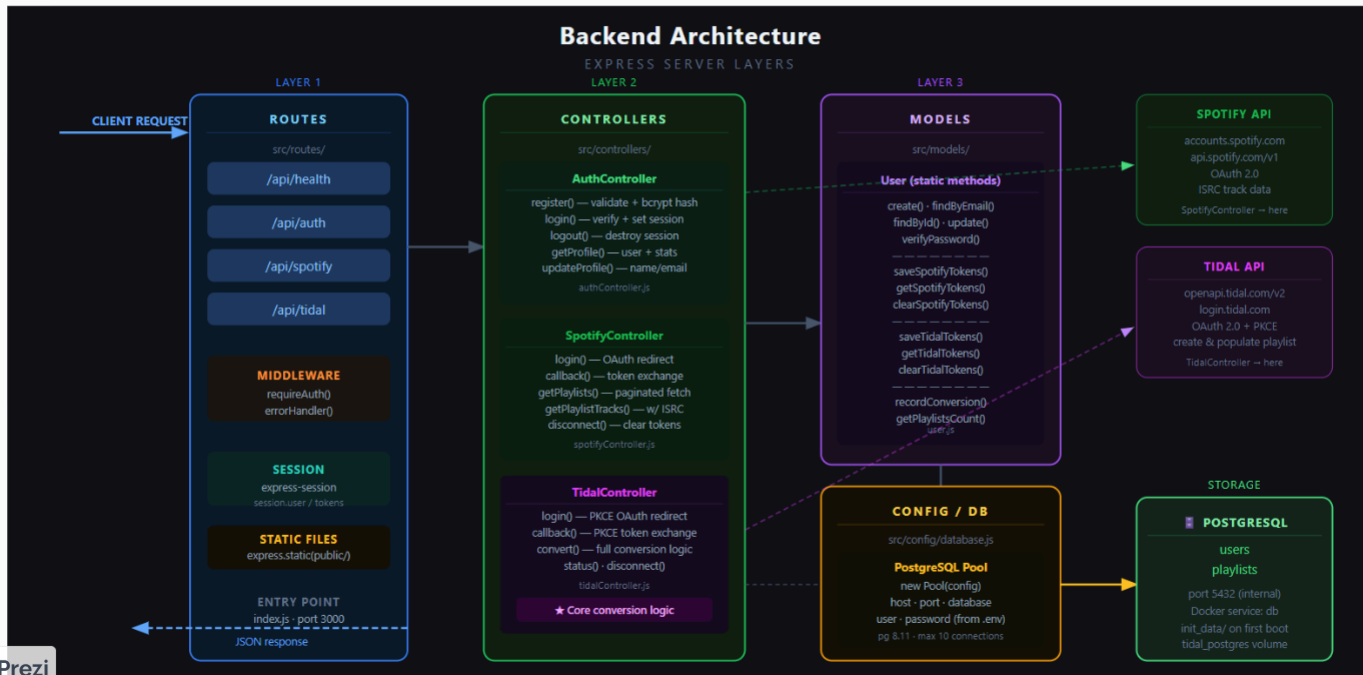


Framework: Express.js (Rating: 4/5) - For
building robust web applications with Node.js.

System Architecture



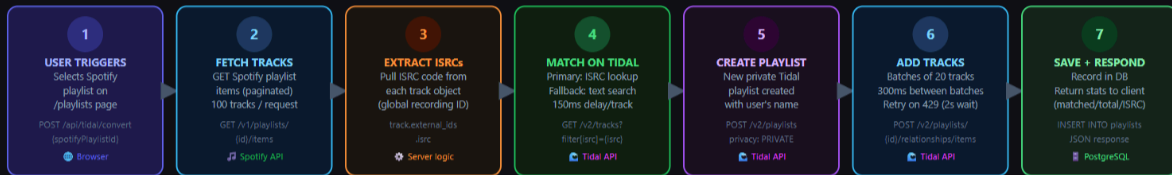
Backend Architecture



Conversion Flow

Playlist Conversion Flow

POST /api/tidal/convert — TidalController.convert()



STEP 4 DETAIL — Track Matching Strategy

Phase A — ISRC Lookup (Primary)

GET /v2/tracks?filter[isrc]=<isrc>&countryCode=US

✓ Exact match — highest accuracy

International Standard Recording Code
Globally unique per recording
Same song on Spotify and Tidal share ISRC
Works across regional variants

If no match → fallback to Phase B

150ms delay between each track search

Phase B — Full-Text Search (Fallback)

Cleanup heuristics applied first:

Remove "(feat. X)" / "(ft. X)" notations

Remove "Radio Mix", "Remaster" suffixes

Search order (first match wins):

1. Full track name + artist
2. Cleaned name + artist
3. Cleaned name only
4. Full track name only
5. Artist name only (last resort)

Top 5 results returned → use [0]

Response Payload

```
{
  "success" : boolean,
  "matched": {
    "album": {
      "byIsrc": {
        "isrc": number,
        "bySearch": {
          "isrc": number,
          "total": {
            "spotify": {
              "withIsrc": {
                "tidalPlaylistId" : string,
                "unmatched": {
                  "isrc": string
                }
              }
            }
          }
        }
      }
    }
  }
}
```

RATE LIMITING & THROTTLING

- 150ms delay between individual Tidal track searches
- 300ms delay between each batch of 20 tracks added to the playlist
- On HTTP 429 (Too Many Requests): wait 2 seconds then retry once

TOKEN AUTO-REFRESH

- Tokens checked before every API call
- Refresh if expires_at < Date.now() + 60,000ms
- New tokens immediately persisted to PostgreSQL



Authentication



Challenges & Resolutions



Platform Redirect URL: Provided white-listing for Spotify API automatically helping with access for different accounts.



Docker Issues: issues with the client-side Docker application and complications with the containers; solved by reconfiguring docker and NodeJS



API Rate Limits (100 tracks per call): Solved by spreading out API calls to convert bigger playlists.



Hosting on Render: configuring and connecting the database to the Render application

Future Scope & Enhancements



- ➡ Social Media: Create community through likes and creating comments on converted playlists
- ➡ Additional Streaming Services: Expand compatibility to include other major music streaming platforms like Apple Music and YouTube Music.
- ➡ Randomized Playlists: Introduce a fun option for expanding users musical palate through randomized playlists.
- ➡ User Analytics & Insights: Provide users with data insights into their listening habits across platforms after migration.

Live Project Demonstration

Experience the Spotify to Tidal Converter firsthand. Our live demonstration will walk you through the entire migration process, showcasing user authentication, playlist selection, conversion, and successful transfer.

**Liked Songs**
Spotify · 248 tracks

**Blinding Lights**
The Weeknd3:20

**Levitating**
Dua Lipa3:23

**As It Was**
Harry Styles2:37

Converting →

**Liked Songs**
Tidal · 248 tracks ✓Done

**Blinding Lights**
The Weeknd3:20

**Levitating**
Dua Lipa3:23

**As It Was**
Harry Styles2:37